Gary Hobson

MAT 260 Module 6

June 9, 2025

1 11.6-2

(1) Why shouldn't Bob believe Alice even if $T = C \oplus K$?
Because Alice can choose K after the World Cup ends. The one time pad encryption is:

$$C = T \oplus K \quad \Rightarrow \quad T = C \oplus K$$

Since both T and K are arbitrary bit strings of the same length, Alice can fabricate any message T she wants after the fact by selecting:

$$K = C \oplus T$$

This makes the scheme completely non binding she could claim to have predicted any outcome and produce a K that makes it look valid.

(2) How does $H(K)$ help?
By sending a cryptographic hash $H(K)$ of the key before the outcome is known, Alice commits to a specific key K. Cryptographic hashes are:

- **Preimage resistant:** Given $H(K)$, it is infeasible to find K.
- **Collision resistant:** It is infeasible to find two values K and K' such that $H(K) = H(K')$.

So once Alice sends both C and $H(K)$, she cannot change K later without being detected. When she reveals K after the World Cup, Bob checks that $H(K)$ matches what she sent earlier. This gives her prediction some integrity.

Summary: $H(K)$ acts as a commitment to K, locking Alice into her original guess.

(3) How can Bob determine T using just C and $H(K)$?
This is a clever twist. Bob does not know K, but the space of possible T (World Cup winners) is small under 100 teams. So Bob can:

- For each possible team name T_i:
  - Compute $K_i = C \oplus T_i$.
  - Hash $K_i \to H(K_i)$.
  - Check if $H(K_i)$ matches the $H(K)$ Alice sent.
- If he finds a match, that T_i must be Alice's original prediction.

Summary: Since the set of T is small, Bob can brute force all options and identify the only T such that $H(C \oplus T) = H(K)$.

11.6-6

1. Suppose Eve is able to look at the file. What property of the hash function prevents Eve from finding a password that will be accepted as valid?
Answer:
Pre image resistance prevents Eve from finding a valid password.
This means that given a hash value $H(\text{pwd})$, it is computationally infeasible to reverse engineer or guess the original password pwd that produced it. Even if Eve sees the hash in the file, a good hash function ensures she cannot recover the original password.

2. When the user logs in, and the hash of the user's password matches the hash stored in the file, what property of the hash function says that the user probably entered the correct password?
Answer:
Collision resistance ensures that the correct password was probably entered. This means it is infeasible to find two different inputs pwd1 $\neq$ pwd2 such that $H(\text{pwd1}) = H(\text{pwd2})$. So, if the hash of the entered password matches the stored hash, it is highly likely the user entered the original password and not a different one that just happens to hash to the same value.

11.6-10

1. Why is the professor confident if a student gets 100%?
A cryptographic hash function is designed to be collision resistant and to avalanche, meaning small changes in input cause large, unpredictable changes in output.
So, if a student's hash matches the hash of the correct answer in all 100 bits, the probability of this happening by guessing is 1 in 2^{100} which is astronomically low.
Therefore, the professor is justified in believing the student had the exact correct answer.

This relies on the preimage resistance and unpredictability properties of good cryptographic hash functions.

2. What's the expected score if every student guesses wrong but all have different answers?
If each student submits a completely wrong and unique answer, then each hashed response is essentially a random 100 bit string.
When comparing each random hash to the correct one, the expected number of matching bits is 50 out of 100.
This is because each bit has a 50% chance of being the same between two independently random 100 bit strings.

So, the average score would be approximately 50 for a totally incorrect answer set.

12.8-2

If there are:

$$N = 2^{100} \approx 10^{30} \text{ total possible sequences (unique coin flips)},$$

$$r = 10^{10} \text{ people (each doing a 100 coin flip)},$$

then the probability that no two people share the same sequence is approximately:

$$P(\text{no match}) \approx e^{-\frac{r^2}{2N}}$$

Therefore, the probability of at least one match is:

$$P(\text{match}) \approx 1 - e^{-\frac{r^2}{2N}}$$

Substitute values:

$$r = 10^{10}, \quad N = 10^{30} \quad \Rightarrow \quad \frac{r^2}{2N} = \frac{10^{20}}{2 \cdot 10^{30}} = 5 \times 10^{-11}$$

$$P(\text{match}) \approx 1 - e^{-5\times 10^{-11}} \approx 5 \times 10^{-11}$$

Final Answer:
The probability that two people out of 10^{10} will flip the same 100 coin sequence is approximately:

$$5.0 \times 10^{-11},$$

accurate to two decimal places (i.e., 0.00). This means the probability is essentially negligible.

12.8-4

if Alice encrypts using double encryption:

$$c = E_{K_1}(E_{K_2}(m)).$$

Eve has a known plaintext ciphertext pair (m, c).

Eve computes:

- $E_K(m)$ for $3 \cdot 2^{N/2}$ random keys K,
- $D_L(c)$ for $3 \cdot 2^{N/2}$ random keys L.

(a) Why is there a good chance Eve finds a key pair (L, K) such that $c = E_L(E_K(m))$?

Because of the Birthday Paradox. With roughly $2^{N/2}$ entries in both lists, there is a high probability of a match between one of the computed $E_K(m)$ and one of the $D_L(c)$ values.

The odds favor at least one collision due to the reduced space 2^N, enabling Eve to detect:

$$E_K(m) = D_L(c),$$

which implies:

$$c = E_L(E_K(m)).$$

(b) Why is it unlikely that (L, K) is the correct key pair?

There are many key pairs that can produce the same result due to the nature of the Meet in the Middle (MitM) attack.

Just because there is a match does not confirm correctness. The chance of a false positive is high unless Eve has multiple plaintext ciphertext pairs to filter out incorrect key pairs.

(c) What is the difference between this and a traditional Meet in the Middle attack?

In a traditional MitM attack, all keys are tried exhaustively, creating complete forward and backward tables.

In this Birthday style attack, the lists are created from randomly selected keys, not all keys, meaning it is probabilistic, not guaranteed.

It is faster but less reliable optimized to exploit birthday collisions, not exhaustive search.

12.8-11

If a hash function outputs only 50 bits, then using the birthday paradox, the somebody only needs to create about $2^{25} \approx 33.5$ million variants of two documents to have a high chance of a collision.

By generating small changes in a document (like changing punctuation or spacing), Eve can produce 2^{30} variants of both the real and fraudulent documents.

Once a collision is found (two documents with the same hash), Eve gets Bob to sign the legitimate one and then transfers the signature to the fraudulent one.

So,

Exercise Answers Cross Checked
Why can Eve forge the signature?
Because the hash output is only 50 bits, Eve can use a birthday attack to generate two documents (one benign and one malicious) that hash to the same value. With 10^9 known signed documents and the ability to do 2^{30} computations, Eve can search for or construct a hash collision efficiently.

How does Eve execute the forgery?
She modifies M into many versions (harmless), finds one that hashes to the same value as her malicious version, gets Bob to sign the benign one, and reuses the signature for her document since it has the same hash.